JavaScript is disabled on your browser.

compact1, compact2, compact3

java.time.temporal

Interface TemporalAccessor

All Known Subinterfaces:

ChronoLocalDate, ChronoLocalDateTime<D>, ChronoZonedDateTime<D>, Era, Temporal

All Known Implementing Classes:

DayOfWeek, HijrahDate, HijrahEra, Instant, IsoEra, JapaneseDate, JapaneseEra, LocalDate, LocalDateTime, LocalTime, MinguoDate, MinguoEra, Month, MonthDay, OffsetDateTime, OffsetTime, ThaiBuddhistDate, ThaiBuddhistEra, Year, YearMonth, ZonedDateTime, ZoneOffset

---

```
public interface TemporalAccessor
```

Framework-level interface defining read-only access to a temporal object, such as a date, time, offset or some combination of these.

This is the base interface type for date, time and offset objects. It is implemented by those classes that can provide information as fields or queries.

Most date and time information can be represented as a number. These are modeled using `TemporalField` with the number held using a `long` to handle large values. Year, month and day-of-month are simple examples of fields, but they also include instant and offsets. See `ChronoField` for the standard set of fields.

Two pieces of date/time information cannot be represented by numbers, the chronology and the time-zone. These can be accessed via queries using the static methods defined on `TemporalQuery`.

A sub-interface, `Temporal`, extends this definition to one that also supports adjustment and manipulation on more complete temporal objects.

This interface is a framework-level interface that should not be widely used in application code. Instead, applications should create and pass around instances of concrete types, such as `LocalDate`. There are many reasons for this, part of which is that implementations of this interface may be in calendar systems other than ISO. See `ChronoLocalDate` for a fuller discussion of the issues.

Implementation Requirements:

`This interface places no restrictions on the mutability of implementations, however immutability is strongly recommended.`

Since:

`1.8`

Method Summary

Abstract Methods | **Default Methods**

Modifier and Type	Method	Description
`default int`	**`get`**`(`**`TemporalField`** `field)`	Gets the value of the specified field as an `int`.
`long`	**`getLong`**`(`**`TemporalField`** `field)`	Gets the value of the specified field as a `long`.
`boolean`	**`isSupported`**`(`**`TemporalField`** `field)`	Checks if the specified field is supported.
`default <R> R`	**`query`**`(`**`TemporalQuery`**`<R> query)`	Queries this date-time.
`default` **`ValueRange`**	**`range`**`(`**`TemporalField`** `field)`	Gets the range of valid values for the specified field.

Method Detail

isSupported

```
boolean isSupported(TemporalField field)
```

Checks if the specified field is supported.

This checks if the date-time can be queried for the specified field. If false, then calling the `range` and `get` methods will throw an exception.

Implementation Requirements:

Implementations must check and handle all fields defined in `ChronoField`. If the field is supported, then true must be returned, otherwise false must be returned.

If the field is not a ChronoField, then the result of this method is obtained by invoking TemporalField.isSupportedBy(TemporalAccessor) passing this as the argument.

Implementations must ensure that no observable state is altered when this read-only method is invoked.

Parameters:

field - the field to check, null returns false

Returns:

true if this date-time can be queried for the field, false if not

range

```
default ValueRange range(TemporalField field)
```

Gets the range of valid values for the specified field.

All fields can be expressed as a `long` integer. This method returns an object that describes the valid range for that value. The value of this temporal object is used to enhance the accuracy of the returned range. If the date-time cannot return the range, because the field is unsupported or for some other reason, an exception will be thrown.

Note that the result only describes the minimum and maximum valid values and it is important not to read too much into them. For example, there could be values within the range that are invalid for the field.

Implementation Requirements:

Implementations must check and handle all fields defined in `ChronoField`. If the field is supported, then the range of the

field must be returned. If unsupported, then an UnsupportedTemporalTypeException must be thrown.

If the field is not a ChronoField, then the result of this method is obtained by invoking TemporalField.rangeRefinedBy(TemporalAccessorl) passing this as the argument.

Implementations must ensure that no observable state is altered when this read-only method is invoked.

The default implementation must behave equivalent to this code:

```
  if (field instanceof ChronoField) {
    if (isSupported(field)) {
      return field.range();
    }
    throw new UnsupportedTemporalTypeException("Unsupported field: " + field);
  }
  return field.rangeRefinedBy(this);
```

Parameters:

field - the field to query the range for, not null

Returns:

the range of valid values for the field, not null

Throws:

DateTimeException - if the range for the field cannot be obtained

UnsupportedTemporalTypeException - if the field is not supported

get

```
default int get(TemporalField field)
```

Gets the value of the specified field as an int.

This queries the date-time for the value of the specified field. The returned value will always be within the valid range of values for the field. If the date-time cannot return the value, because the field is unsupported or for some other reason, an exception will be thrown.

Implementation Requirements:

Implementations must check and handle all fields defined in ChronoField. If the field is supported and has an int range, then the value of the field must be returned. If unsupported, then an UnsupportedTemporalTypeException must be thrown.

If the field is not a ChronoField, then the result of this method is obtained by invoking TemporalField.getFrom(TemporalAccessor) passing this as the argument.

Implementations must ensure that no observable state is altered when this read-only method is invoked.

The default implementation must behave equivalent to this code:

```
  if (range(field).isIntValue()) {
    return range(field).checkValidIntValue(getLong(field), field);
  }
  throw new UnsupportedTemporalTypeException("Invalid field " + field + " + for g
```

Parameters:

field - the field to get, not null

Returns:

the value for the field, within the valid range of values

Throws:

DateTimeException - if a value for the field cannot be obtained or the value is outside the range of valid values for the field

UnsupportedTemporalTypeException - if the field is not supported or the range of values exceeds an int

ArithmeticException - if numeric overflow occurs

getLong

```
long getLong(TemporalField field)
```

Gets the value of the specified field as a long.

This queries the date-time for the value of the specified field. The returned value may be outside the valid range of values for the field. If the date-time cannot return the value, because the field is unsupported or for some other reason, an exception will be thrown.

Implementation Requirements:

Implementations must check and handle all fields defined in ChronoField. If the field is supported, then the value of the field must be returned. If unsupported, then an UnsupportedTemporalTypeException must be thrown.

If the field is not a ChronoField, then the result of this method is obtained by invoking TemporalField.getFrom(TemporalAccessor) passing this as the argument.

Implementations must ensure that no observable state is altered when this read-only method is invoked.

Parameters:

field - the field to get, not null

Returns:

the value for the field

Throws:

DateTimeException - if a value for the field cannot be obtained

UnsupportedTemporalTypeException - if the field is not supported

ArithmeticException - if numeric overflow occurs

query

```
default <R> R query(TemporalQuery<R> query)
```

Queries this date-time.

This queries this date-time using the specified query strategy object.

Queries are a key tool for extracting information from date-times. They exists to externalize the process of querying, permitting different approaches, as per the strategy design pattern. Examples might be a query that checks if the date is the day before February 29th in a leap year, or calculates the number of days to your next birthday.

The most common query implementations are method references, such as `LocalDate::from` and `ZoneId::from`. Additional implementations are provided as static methods on TemporalQuery.

Implementation Requirements:

The default implementation must behave equivalent to this code:

```
  if (query == TemporalQueries.zoneId() ||
        query == TemporalQueries.chronology() || query == TemporalQueries.precisi
    return null;
  }
  return query.queryFrom(this);
```

Future versions are permitted to add further queries to the if statement.

All classes implementing this interface and overriding this method must call TemporalAccessor.super.query(query). JDK classes may avoid calling super if they provide behavior equivalent to the default behaviour, however non-JDK classes may not utilize this optimization and must call super.

If the implementation can supply a value for one of the queries listed in the if statement of the default implementation, then it must do so. For example, an application-defined HourMin class storing the hour and minute must override this method as follows:

```
if (query == TemporalQueries.precision()) {
  return MINUTES;
}
return TemporalAccessor.super.query(query);
```

Implementations must ensure that no observable state is altered when this read-only method is invoked.

Type Parameters:

R - the type of the result

Parameters:

query - the query to invoke, not null

Returns:

the query result, null may be returned (defined by the query)

Throws:

DateTimeException - if unable to query

ArithmeticException - if numeric overflow occurs

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.